

NATIONAL ECONOMICS UNIVERSITY  
FACULTY OF DATA SCIENCE & AI

INSTRUCTOR MARKING SCHEME  
Academic Year: 2026–2027 Semester: Fall 2026

## ASSIGNMENT – OFFICIAL MARKING SCHEME

Course: Data Structures and Algorithms with Python (DSAI1002)

Topic: Sorting, Selection, and Searching | Paper Code: 102 | Total: 10.0 Points

### Examiner Instructions & General Grading Guidelines:

- Mark on a strict scale of **10.0 points**. Fractional grading in increments of **0.25 points** is permissible.
- In algorithmic execution traces, award partial credit for correct initial passes even if an arithmetic or copying slip occurs later.
- For asymptotic derivations and formal proofs, penalize unsupported assertions where explicit steps or invariants are required.

### Part I: Conceptual Foundations & Theoretical Principles (3.0 Points)

#### Question 1: Sorting Classification, Stability & Memory Overhead (1.0 Point)

- **(a) Stability Definition, Multi-Column Sorting & Selection Sort [0.5 pts]:**
  - **Formal Definition [0.15 pts]:** A sorting algorithm is *stable* if equal elements retain their original relative order in the output. Formally, if  $A[i] = A[j]$  with original indices  $i < j$ , then their sorted indices satisfy  $\pi(i) < \pi(j)$ .
  - **Multi-Column Benefit [0.15 pts]:** Enables multi-criteria sorting by performing successive stable sorts from least-significant to most-significant key (e.g., sort by ‘Name’ first, then stably sort by ‘Department’ to preserve alphabetical name ordering within each department).
  - **Selection Sort Instability [0.2 pts]:** Selection sort is inherently *unstable* because finding the minimum in the unsorted suffix and swapping it into place can jump an element over identical keys. *Counterexample:* Consider  $[2_a, 2_b, 1]$ . Step 1 finds minimum 1 and swaps with index 0, yielding  $[1, 2_b, 2_a]$ . The relative order of  $2_a$  and  $2_b$  is inverted.
- **(b) Memory Overhead of Standard Sorts [0.5 pts]:**
  - **In-Place Definition [0.1 pts]:** An algorithm is in-place if its auxiliary memory requirement is  $O(1)$  (or  $O(\log n)$  call-stack frames), requiring no extra auxiliary buffer proportional to  $n$ .
  - **Merge Sort [0.15 pts]:** Requires  $\Theta(n)$  **auxiliary memory** to maintain temporary merge buffers during merging; it is not in-place on array structures.
  - **Quicksort [0.15 pts]:** In-place on array elements, but requires  $\Theta(\log n)$  **average** /  $\Theta(n)$  **worst-case** auxiliary stack memory for recursive activation frames.
  - **Insertion Sort / Heapsort [0.1 pts]:** Strictly  $O(1)$  **auxiliary space**.

**Question 2: Information-Theoretic Lower Bound for Comparison Sorts (1.0 Point)**• (a) **Decision Tree Model & Leaf Node Bound [0.5 pts]:**

- **Decision Tree Formulation [0.2 pts]:** Every comparison-based sort can be modeled as a binary decision tree where each internal node represents a comparison  $A[i] < A[j]$  with two outgoing branches ('true' or 'false'), and each leaf corresponds to a distinct sorted permutation.
- **Number of Leaves [0.15 pts]:** For  $n$  distinct elements, there are  $n!$  possible permutations. To correctly sort every input permutation, the tree must possess at least  $L \geq n!$  reachable leaf nodes.
- **Tree Height Lower Bound [0.15 pts]:** A binary tree of height  $h$  contains at most  $2^h$  leaves. Therefore:

$$2^h \geq L \geq n! \implies h \geq \lceil \log_2(n!) \rceil.$$

The height  $h$  represents the worst-case number of comparisons.

• (b) **Stirling's Derivation & Non-Comparison Bypass [0.5 pts]:**

- **Logarithmic Sum Derivation [0.3 pts]:**

$$\begin{aligned} \log_2(n!) &= \sum_{i=1}^n \log_2 i \geq \int_1^n \log_2 x \, dx \\ &= [x \log_2 x - x \log_2 e]_1^n = n \log_2 \left(\frac{n}{e}\right) + \log_2 e = \Omega(n \log n). \end{aligned}$$

Hence, any comparison-based sort must execute at least  $\Omega(n \log n)$  comparisons in the worst case.

- **Non-Comparison Bypass [0.2 pts]:** Algorithms such as Counting Sort and Radix Sort achieve  $O(n)$  time by *not* relying on pair-wise comparisons. Instead, they treat keys as numerical values or bit-patterns to perform direct memory addressing (array indexing), bypassing the binary decision tree restriction.

**Question 3: Binary Search Invariants & Integer Overflow (1.0 Point)**• (a) **Midpoint Integer Overflow [0.5 pts]:**

- **The Bug [0.25 pts]:** The standard midpoint calculation `mid = (low + high) // 2` sums two large indices. In fixed-width integer systems (e.g., 32-bit signed integers in C/C++/Java), if  $low + high > 2^{31} - 1 \approx 2.14 \times 10^9$ , the sum overflows to a negative value, causing index-out-of-bounds exceptions or memory segmentation faults (the famous 2006 Java standard library bug).
- **Safe Formula [0.25 pts]:** Use subtraction: `mid = low + (high - low) // 2` (or unsigned shift `(low + high) >> 1`). In this form,  $high - low \geq 0$ , eliminating overflow risk.

• (b) **Modifying Binary Search for First Occurrence [0.5 pts]:**

- **Behavior of Standard Binary Search [0.15 pts]:** Immediately halts upon finding any match  $A[mid] == target$ . With duplicate values, it returns an arbitrary matching index.
- **Modification [0.2 pts]:** When  $A[mid] == target$ , record  $mid$  as a candidate and continue searching left: `ans = mid; high = mid - 1`.
- **Loop Invariant & Complexity [0.15 pts]:** If target exists, its first occurrence lies within  $[low, high]$  or is recorded in `ans`. Search range halves every step, preserving  $O(\log n)$  time and  $O(1)$  space.

## Part II: Algorithmic Traces & Execution Analysis (4.0 Points)

### Question 4: Counting Sort Mechanics & Stability (1.5 Points)

Given array  $A = [4, 2, 2, 8, 3, 3, 1]$  of size  $n = 7$  with keys in range  $[1..8]$ .

• (a) **Frequency Array  $C$  and Prefix Sum Array  $P$  [0.5 pts]:**

– **Frequency Array  $C$  [0.25 pts]:** Size 9 (indices 0..8):

| Index $v$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----------|---|---|---|---|---|---|---|---|---|
| $C[v]$    | 0 | 1 | 2 | 2 | 1 | 0 | 0 | 0 | 1 |

– **Cumulative Prefix Sum Array  $P$  [0.25 pts]:**  $P[v] = \sum_{k=0}^v C[k]$ :

| Index $v$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----------|---|---|---|---|---|---|---|---|---|
| $P[v]$    | 0 | 1 | 3 | 5 | 6 | 6 | 6 | 6 | 7 |

$P[v]$  indicates that elements of value  $v$  occupy output positions up to 1-based rank  $P[v]$  (0-based index  $P[v] - 1$ ).

• (b) **Right-to-Left Placement Trace in Output Array  $B$  [0.5 pts]:** Scanning  $A$  backwards from index 6 to 0 (distinguishing duplicates  $2_a, 2_b$  and  $3_a, 3_b$ ):

- $j = 6, A[6] = 1$ : Target index =  $P[1] - 1 = 1 - 1 = 0$ . Set  $B[0] = 1$ ; decrement  $P[1] = 0$ .
- $j = 5, A[5] = 3_b$ : Target index =  $P[3] - 1 = 5 - 1 = 4$ . Set  $B[4] = 3_b$ ; decrement  $P[3] = 4$ .
- $j = 4, A[4] = 3_a$ : Target index =  $P[3] - 1 = 4 - 1 = 3$ . Set  $B[3] = 3_a$ ; decrement  $P[3] = 3$ .
- $j = 3, A[3] = 8$ : Target index =  $P[8] - 1 = 7 - 1 = 6$ . Set  $B[6] = 8$ ; decrement  $P[8] = 6$ .
- $j = 2, A[2] = 2_b$ : Target index =  $P[2] - 1 = 3 - 1 = 2$ . Set  $B[2] = 2_b$ ; decrement  $P[2] = 2$ .
- $j = 1, A[1] = 2_a$ : Target index =  $P[2] - 1 = 2 - 1 = 1$ . Set  $B[1] = 2_a$ ; decrement  $P[2] = 1$ .
- $j = 0, A[0] = 4$ : Target index =  $P[4] - 1 = 6 - 1 = 5$ . Set  $B[5] = 4$ ; decrement  $P[4] = 5$ .

**Final Array:**  $B = [1, 2_a, 2_b, 3_a, 3_b, 4, 8]$ .

• (c) **Proof of Stability [0.5 pts]:** The prefix sum  $P[v]$  gives the *highest* available slot for key  $v$ . By traversing  $A$  from right to left, the rightmost duplicate encounters  $P[v]$  first and is placed at the larger index. Subsequent duplicates of the same key are placed at sequentially decremented (lower) indices, ensuring earlier arrivals occupy earlier slots. Thus, relative order is strictly preserved.

### Question 5: Lomuto Partitioning & Quicksort Recurrence Trees (1.5 Points)

Given array  $A = [5, 2, 9, 3, 7, 6, 4]$ ,  $low = 0$ ,  $high = 6$ , pivot  $p = A[6] = 4$ .

- **(a) Step-by-Step Lomuto Partition Trace [0.75 pts]:** Initialize  $i = low - 1 = -1$ . Loop variable  $j$  runs from  $low = 0$  to  $high - 1 = 5$ :
  - $j = 0 : A[0] = 5 > 4 \implies$  No swap,  $i = -1$ . Array:  $[5, 2, 9, 3, 7, 6, 4]$ .
  - $j = 1 : A[1] = 2 \leq 4 \implies i = 0$ , swap  $A[i] \leftrightarrow A[j]$  ( $A[0] \leftrightarrow A[1]$ ). Array:  $[2, 5, 9, 3, 7, 6, 4]$ .
  - $j = 2 : A[2] = 9 > 4 \implies$  No swap,  $i = 0$ . Array:  $[2, 5, 9, 3, 7, 6, 4]$ .
  - $j = 3 : A[3] = 3 \leq 4 \implies i = 1$ , swap  $A[i] \leftrightarrow A[j]$  ( $A[1] \leftrightarrow A[3]$ ). Array:  $[2, 3, 9, 5, 7, 6, 4]$ .
  - $j = 4 : A[4] = 7 > 4 \implies$  No swap,  $i = 1$ . Array:  $[2, 3, 9, 5, 7, 6, 4]$ .
  - $j = 5 : A[5] = 6 > 4 \implies$  No swap,  $i = 1$ . Array:  $[2, 3, 9, 5, 7, 6, 4]$ .
  - **Pivot Swap:** Swap  $A[i + 1] \leftrightarrow A[high]$  ( $A[2] \leftrightarrow A[6]$ ): Final partitioned array:  $[2, 3, 4, 5, 7, 6, 9]$  with pivot index  $p = 2$ .

Left partition:  $A[0..1] = [2, 3]$ . Right partition:  $A[3..6] = [5, 7, 6, 9]$ .

- **(b) Best-Case vs. Worst-Case Recurrence Relations [0.75 pts]:**
  - **Best Case [0.35 pts]:** Pivot splits array into two equal halves of size  $\lfloor n/2 \rfloor$ :

$$T(n) = 2T(n/2) + \Theta(n).$$

By Case 2 of the Master Theorem ( $a = 2, b = 2, f(n) = \Theta(n)$ ), tree height is  $\log_2 n$  and work per level is  $\Theta(n)$ , giving  $T(n) = \Theta(n \log n)$ .

- **Worst Case [0.4 pts]:** Array already sorted/reverse-sorted with extreme pivot, yielding partitions of sizes 0 and  $n - 1$ :

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + cn.$$

Expanding the telescoping recurrence:  $T(n) = c \sum_{k=1}^n k = c \frac{n(n+1)}{2} = \Theta(n^2)$ . Recursion tree is completely skewed with height  $n$ .

### Question 6: Quickselect vs. Heap for Top- $k$ Selection (1.0 Point)

Scenario: Finding the top  $k = 50$  largest elements from an unsorted collection of size  $n = 10,000,000$ .

- **(a) Quickselect Mechanics & Pruning [0.5 pts]:**
  - **Pruning Principle [0.25 pts]:** Unlike Quicksort which recurses into both sides, Quickselect checks the partition pivot index  $p$ . If  $p == target$ , it terminates. If  $target < p$ , it recurses *only* into the left subarray; otherwise into the right subarray.
  - **Average Runtime [0.25 pts]:** Because each step discards approximately half the active elements:

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \dots \leq 2n = \Theta(n).$$

- **(b) Min-Heap of Size  $k$  vs. Quickselect Comparison [0.5 pts]:**
  - **Min-Heap of Size  $k$  [0.2 pts]:** Maintain a min-heap of size  $k$ . For each incoming element, compare with heap root ( $O(1)$ ). If larger, replace root in  $O(\log k)$  time. Total time:  $\Theta(n \log k)$ ; Auxiliary Space:  $\Theta(k)$ .
  - **Architectural Decision [0.3 pts]:**
    - \* If data is an **unbounded data stream** or RAM is limited: **Min-Heap of size  $k$**  is strictly required because it processes data on-the-fly using only  $O(k) = O(50)$  auxiliary RAM without storing all  $10^7$  records.
    - \* If all  $n$  elements reside in memory and in-place mutation is allowed: **Quickselect** is faster ( $O(n)$  linear time vs  $O(n \log k)$ ).

### Part III: Algorithmic Design & Problem Solving (3.0 Points)

#### Question 7: Two-Sum on a Sorted Array (1.5 Points)

- (a) **Two-Pointer Algorithm Implementation [0.6 pts]:**
  - Initialize `left = 0` and `right = len(A) - 1`.
  - While `left < right`:
    - \* `curr_sum = A[left] + A[right]`
    - \* If `curr_sum == target`: return `(left + 1, right + 1)` (1-indexed).
    - \* Else if `curr_sum < target`: `left += 1` (need larger sum).
    - \* Else: `right -= 1` (need smaller sum).
  - Return `None` if pointers cross without finding a match.
- (b) **Correctness Proof & Search Space Elimination [0.5 pts]:**
  - **Loop Invariant [0.25 pts]:** If a valid pair  $(i^*, j^*)$  exists, it lies within current index interval  $[left, right]$ .
  - **Pruning Justification [0.25 pts]:**
    - \* When  $A[left] + A[right] < target$ : For any candidate index  $k \leq right$ , because  $A$  is sorted,  $A[left] + A[k] \leq A[left] + A[right] < target$ . Thus,  $A[left]$  cannot form a valid pair with *any* remaining element in the active range. Discarding index  $left$  by advancing `left += 1` is provably safe.
    - \* When  $A[left] + A[right] > target$ : Symmetrically, for any  $k \geq left$ ,  $A[k] + A[right] \geq A[left] + A[right] > target$ . Discarding index  $right$  by retreating `right -= 1` is provably safe.
- (c) **Complexity Derivation [0.4 pts]:**
  - **Time Complexity [0.2 pts]:** Each iteration performs  $O(1)$  operations and decreases the distance  $right - left$  by at least 1. The loop runs at most  $n - 1$  iterations, establishing  $\Theta(n)$  time.
  - **Auxiliary Space [0.2 pts]:** Strictly  $\Theta(1)$  space (only two integer pointer variables), outperforming hash table approaches which consume  $\Theta(n)$  auxiliary memory.

### Question 8: Searching in a Rotated Sorted Array (1.5 Points)

- (a) **Binary Search Algorithm for Rotated Array [0.7 pts]:**
  - Set  $low = 0$ ,  $high = \text{len}(A) - 1$ .
  - While  $low \leq high$ :
    - \* Compute  $mid = low + (high - low) // 2$ .
    - \* If  $A[mid] == \text{target}$ : return  $mid$ .
    - \* **Check if left half is sorted ( $A[low] \leq A[mid]$ ):**
      - If  $A[low] \leq \text{target} < A[mid]$ :  $high = mid - 1$  (target in sorted left half).
      - Else:  $low = mid + 1$  (search right).
    - \* **Otherwise right half must be sorted ( $A[mid] < A[high]$ ):**
      - If  $A[mid] < \text{target} \leq A[high]$ :  $low = mid + 1$  (target in sorted right half).
      - Else:  $high = mid - 1$  (search left).
  - Return -1 if not found.
- (b) **Execution Trace on  $A = [6, 7, 1, 2, 3, 4, 5]$  with Target 1 [0.5 pts]:**
  - **Iteration 1 [0.2 pts]:**  $low = 0, high = 6 \implies mid = 3$  ( $A[3] = 2$ ).  $A[0] = 6 > A[3] = 2 \implies$  Left half is unsorted; right half  $A[3..6] = [2, 3, 4, 5]$  is sorted. Test target: is  $1 \in (2, 5]$ ? False ( $1 < 2$ ). Discard right half:  $high = mid - 1 = 2$ .
  - **Iteration 2 [0.2 pts]:**  $low = 0, high = 2 \implies mid = 1$  ( $A[1] = 7$ ).  $A[0] = 6 \leq A[1] = 7 \implies$  Left half  $A[0..1] = [6, 7]$  is sorted. Test target: is  $1 \in [6, 7]$ ? False ( $1 < 6$ ). Discard left half:  $low = mid + 1 = 2$ .
  - **Iteration 3 [0.1 pts]:**  $low = 2, high = 2 \implies mid = 2$  ( $A[2] = 1$ ).  $A[2] == 1 == \text{target} \implies$  **Match found! Return index 2.**
- (c) **Sorted-Half Invariant & Logarithmic Time Proof [0.3 pts]:** A rotated array sorted originally has at most one inflection point where  $A[k] > A[k + 1]$ . This inflection point can lie in at most one of the two halves  $[low..mid]$  or  $[mid..high]$ . Therefore, at least one half is always monotonically ordered. By testing whether the target falls within the sorted range, we discard half the elements every iteration:

$$T(n) = T(n/2) + O(1) = \Theta(\log n), \quad \text{Auxiliary Space} = O(1).$$

### End of Official Marking Scheme – Paper Code 102

Department of Data Science & AI · National Economics University